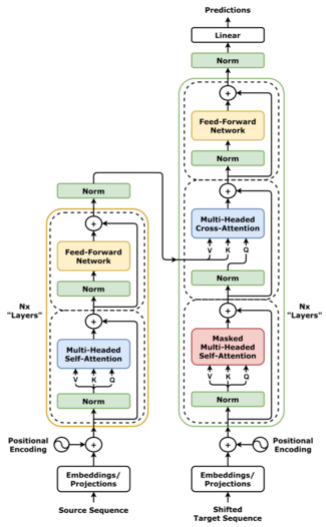
Трансформатор (глубокое обучение)

В глубоком обучении**трансформер** — это семейство архитектур искусственных нейронных сетей, основанных на механизме многоголового внимания, в котором входные данные, такие как текст, изображения или аудио, преобразуются в последовательность числовых представлений, называемых токенами, а каждый токен преобразуется в вектор с помощью слоя внедрения.[1] На каждом уровне каждый токенконтекстуализируется в рамках контекстного окна с другими (незамаскированными) токенами с помощью параллельного механизма многоголового внимания, что позволяет усилить сигнал для ключевых токенов и ослабить сигнал для менее важных. Поскольку механизм внутреннего внимания сам по себе инвариантен к перестановке, трансформеры добавляют позиционную информацию, обычно с помощью позиционных кодировок или изученных позиционных вложений, поэтому порядок токенов может влиять на результат. [1]



Стандартная архитектура трансформера: слева — кодировщик, справа — декодировщик. Примечание: здесь используется соглашение о префиксном обозначении, которое отличается от соглашения о постфиксном обозначении, использованного в оригинальном трансформере 2017 года.

Поскольку трансформеры не обрабатывают токены по одному, их вычисления можно распараллелить по позициям в последовательности во время обучения, в отличие от рекуррентных нейронных сетей, таких как долгая краткосрочная память (LSTM).[2] Более поздние варианты широко используются для обучения больших языковых моделей (БЯМ) на больших (языковых) наборах данных.

Первоначальная версия архитектуры трансформера была предложена в статье 2017 года «Attention Is All You Need" исследователями из Google.[1] Предшественники трансформеров были разработаны как усовершенствованная версия предыдущих архитектур для машинного перевода,[3][4] но с тех пор нашли множество применений. Они используются в крупномасштабной обработке естественного языка, компьютерном зрении (трансформеры для компьютерного зрения), обучении с подкреплением,[5] аудио,[6] мультимодальном обучении, робототехнике,[7] и игре в шахматы.[8] Это также привело к разработке предварительно обученных систем, таких как генеративные предварительно обученные трансформеры (GPT).[9]

История

Предшественники

До появления трансформеров для моделирования и генерации последовательностей широко использовались рекуррентные нейронные сети (РНС). Одним из первых примеров, получивших широкое распространение, была сеть Элмана (1990). Теоретически информация от одного токена может распространяться по всей последовательности, но на практике из-за проблемы исчезающего градиента состояние модели в конце длинного предложения не содержит точной информации о предшествующих токенах, которую можно было бы извлечь.

Ключевым прорывом стала LSTM (первоначально описанная в техническом отчете 1995 года и официально опубликованная в 1997 году)[2][примечание 1] — рекуррентная нейронная сеть, в которой были внедрены механизмы гейтинга для решения проблемы исчезающего градиента, что позволило эффективно обучать моделированию длинных последовательностей. Одним из ключевых архитектурных элементов стало использование *мультипликативных гейтинговых блоков*, в которых выходные данные одних нейронов модулируют выходные данные других. Эти мультипликативные блоки концептуально отличаются от механизма аддитивного внимания, который позже был внедрен в модели типа «последовательность — последовательность». [10] Нейронные сети, использующие мультипликативные блоки, позже стали называть *сигма-пи-сетями*[11] или *сетями высшего порядка*.[12] LSTM стали стандартной архитектурой для моделирования длинных последовательностей вплоть до публикации трансформеров в 2017 году. Однако LSTM, как и большинство других рекуррентных нейронных сетей, по-прежнему использует последовательную обработку.[примечание 2] В частности, рекуррентные нейронные сети обрабатывают по одному токену за раз, от первого до последнего; они не могут обрабатывать все токены в последовательности параллельно.

Трансформеры позволяют параллельно обрабатывать токены во время обучения, но стандартное самовнимание требует вычислительных ресурсов и памяти, которые растут квадратично в зависимости от длины последовательности.

Внимание: seq2seq

Идея преобразования последовательностей с помощью кодера и декодера была разработана в начале 2010-х годов. В качестве первоисточников, на основе которых была создана модель seq2seq, обычно называют две одновременно опубликованные статьи 2014 года. [13][14]

Модель машинного перевода с 380 миллионами параметров использует две долговременные и кратковременные памяти (LSTM).[14] Ее архитектура состоит из двух частей. *Кодировщик* — это LSTM, которая принимает последовательность токенов и преобразует ее в вектор. *Декодировщик* — это еще одна LSTM, которая преобразует вектор в последовательность токенов. Аналогичным образом в другой модели с 130 миллионами параметров вместо LSTM использовались управляемые рекуррентные блоки (GRU).[13] Более поздние исследования показали, что GRU не лучше и не хуже LSTM для seq2seq.[15][16]

These early seq2seq models had no attention mechanism, and the state vector is accessible only after the *last* word of the source text was processed. Although in theory such a vector retains the information about the whole original sentence, in practice the information is poorly preserved. This is because the input is processed sequentially by one recurrent network into a *fixed*-size output vector, which is then processed by another recurrent network into an output. If the input is long, then the output vector would not be able to contain all relevant information, degrading the output. As evidence, reversing the input sentence improved seq2seq translation.[17]

The *RNN search* model introduced an attention mechanism to seq2seq for machine translation to solve the bottleneck problem (of the *fixed-size* output vector), allowing the model to process long-distance dependencies more easily. The name is because it "emulates searching through a source sentence during decoding a translation".[3]

The relative performances were compared between global (that of *RNN search*) and local (sliding window) attention model architectures for machine translation, finding that mixed attention had higher quality than global attention, while local attention reduced translation time.[18]

In 2016, Google Translate was revamped to Google Neural Machine Translation, which replaced the previous model based on statistical machine translation. The new model was a seq2seq model where the encoder and the decoder were both 8 layers of bidirectional LSTM.[19] It took

nine months to develop, and it outperformed the statistical approach, which took ten years to develop.[20]

Parallelizing attention

Seq2seq models with attention (including self-attention) still suffered from the same issue with recurrent networks, which is that they are hard to parallelize, which prevented them from being accelerated on GPUs. In 2016, *decomposable attention* applied a self-attention mechanism to feedforward networks, which are easy to parallelize, and achieved state-of-the-art (SOTA) results in textual entailment with an order of magnitude fewer parameters than LSTMs.[21] One of its authors, Jakob Uszkoreit, suspected that attention *without* recurrence would be sufficient for language translation, thus the title "attention is *All* you need".[22] That hypothesis was against conventional wisdom at the time, and even his father Hans Uszkoreit, a well-known computational linguist, was skeptical.[22] In the same year, self-attention (called *intra-attention or intra-sentence attention*) was proposed for LSTMs.[23]

On 2017-06-12, the original (100M-parameter) encoder–decoder transformer model was published in the "Attention is All you need" paper. At the time, the focus of the research was on improving seq2seq for machine translation, by removing its recurrence to process all tokens in parallel, but preserving its dot-product attention mechanism to keep its text processing performance.[1] This led to the introduction of a multi-head attention model that was easier to parallelize due to the use of independent heads and the lack of recurrence. Its parallelizability was an important factor to its widespread use in large neural networks.[24]

AI boom era

In spring 2017, before the "Attention is All you need" preprint was published, one of the co-authors applied the "decoder-only" variation of the architecture to generate fictitious Wikipedia articles.[25] Transformer architecture is now used alongside many generative models that contribute to the ongoing AI boom.

The "reference implementation" of the original Transformer was written in a TensorFlow library.[26][27] In language modelling, ELMo (2018) was a bi-directional LSTM that produces contextualized word embeddings, improving upon the line of research from bag of words and word2vec. It was followed by BERT (2018), an encoder-only transformer model.[28] In October 2019, Google started using BERT to process search queries.[29] In 2020, Google Translate replaced the previous RNN-encoder–RNN-decoder model by a transformer-encoder–RNN-decoder model.[30]

Starting in 2018, the OpenAI GPT series of decoder-only transformers became state of the art in natural language generation. At the end of 2022, ChatGPT, a chatbot based on a fine-tuned

variant of GPT-3.5, became unexpectedly[31][32] popular, triggering a boom around large language models.[33]

Transformers have been applied in modalities beyond text. Four days after the publication of "Attention is All You Need", a multimodal transformer architecture, MultiModel, was published by most authors of that paper.[34] Other examples include the vision transformer, which has led to developments in convolutional neural networks.[35] Image and video generators like DALL-E (2021), Stable Diffusion 3 (2024), and Sora (2024), use transformers to analyse input data (like text prompts) by breaking it down into "tokens" and then calculating the relevance between each token using self-attention, which helps the model simulate the context and relationships within the data.

Обучение

Методы стабилизации тренировок

Простая архитектура трансформера с трудом сходилась. В оригинальной статье[1] авторы рекомендовали использовать прогрев скорости обучения. То есть скорость обучения должна линейно увеличиваться от 0 до максимального значения в течение первой части обучения (обычно рекомендуется, чтобы она составляла 2 % от общего количества шагов обучения), а затем снова снижаться.

В статье 2020 года было показано, что использование нормализации слоя *до* (а не после) многоголового внимания и слоев прямой связи стабилизирует процесс обучения и не требует прогревания скорости обучения. [36] Это «трансформер до нормализации слоя», который используется чаще, чем оригинальный «трансформер после нормализации слоя».

Регуляризация

В исходном трансформере для уменьшения переобучения использовался дропаут. Дропаут применялся к выходным данным каждого подслоя перед его остаточным соединением и нормализацией, а также к суммам вложений токенов и позиционных кодировок. В исходной статье также использовалось сглаживание меток, при котором определенное целевое распределение заменяется распределением, в котором альтернативным токенам присваивается небольшая вероятность. Эти методы могут улучшить обобщение за счет снижения чрезмерной уверенности в обучающих целях. [37]

Распределенное обучение

Большие трансформерные модели обычно обучаются на нескольких графических процессорах или других ускорителях, поскольку их параметры, активации и обучающие

данные могут не уместиться на одном устройстве. При параллельной обработке данных модель копируется на несколько устройств, а между ними распределяются разные пакеты обучающих примеров; затем перед обновлением параметров синхронизируются градиенты. При параллельной обработке модели сама модель распределяется между устройствами. При тензорном параллелизме операции внутри отдельных слоев распределяются между устройствами, а при конвейерном параллелизме последовательные группы слоев распределяются между разными устройствами. [38]

Предварительное обучение и тонкая настройка

Трансформеры обычно сначала проходят предварительное обучение с помощью самоконтролируемого обучения на большом общем наборе данных, а затем контролируемую тонкую настройку на небольшом наборе данных для конкретной задачи. Набор данных для предварительного обучения обычно представляет собой большой немаркированный корпус, например The Pile. Задачи для предварительного обучения и тонкой настройки обычно включают:

- понимание прочитанного
- анализ тональности[1]
- перефразирование[1]

В отчете о трансформере T5 [39] описано большое количество задач по предварительной подготовке к работе с естественным языком. Вот некоторые примеры:

- restoring or repairing incomplete or corrupted text. For example, the input, *"Thank you ~~ me to your party ~~ week"*, might generate the output, *"Thank you **for inviting** me to your party **last** week"*.
- translation between natural languages (machine translation)
- judging the pragmatic acceptability of natural language. For example, the following sentence might be judged "not acceptable",[40] because even though it is syntactically well-formed, it is improbable in ordinary human usage: *The course is jumping well.*

While each of these tasks is trivial or obvious for human native speakers of the language (or languages), they have typically proved challenging for previous generations of machine learning architecture.

Knowledge distillation

Knowledge distillation is a method for training a smaller model, called a "student", to reproduce the output behavior of a larger "teacher" model. In transformer models, the student can be trained using the teacher's output probability distribution, intermediate representations, or attention patterns in addition to the original training labels. Distillation can lower computational

and memory requirements of deployment while keeping most of the teacher model's performance on a specified task.

Tasks

In general, there are three classes of language modelling tasks: "masked",[41] "autoregressive",[42] and "prefixLM".[43] These classes are independent of a specific modeling architecture such as transformer, but they are often discussed in the context of transformer.

In a masked task,[41] one or more of the tokens is masked out, and the model would produce a probability distribution predicting what the masked-out tokens are based on the context. The loss function for the task is typically sum of log-perplexities for the masked-out tokens: and the model is trained to minimize this loss function. The BERT series of models are trained for masked token prediction and another task. ("Masked" as in "masked language modelling" is not "masked" as in "masked attention".)

In an autoregressive task,[42] the entire sequence is masked at first, and the model produces a probability distribution for the first token. Then the first token is revealed and the model predicts the second token, and so on. The loss function for the task is still typically the same. The GPT series of models are trained by autoregressive tasks.

In a prefixLM task,[43] the sequence is divided into two parts. The first part is presented as context, and the model predicts the first token of the second part. Then that would be revealed, and the model predicts the second token, and so on. The loss function for the task is still typically the same. The T5 series of models are trained by prefixLM tasks. ("PrefixLM" as in "prefix language modeling" is not "prefixLM" as in "prefix language model".)

Архитектура

Все трансформаторы имеют одинаковые первичные компоненты:

- Токенизаторы, которые преобразуют текст в токены.
- Слой внедрения, который преобразует токены и их позиции в векторные представления.
- Слои трансформера, которые выполняют повторяющиеся преобразования векторных представлений, извлекая все больше и больше лингвистической информации. Они состоят из чередующихся слоев внимания и слоев прямой передачи. Существует два основных типа слоев трансформера: слои кодировщика и слои декодера, а также их различные варианты.
- Слой извлечения, который преобразует конечные векторные представления обратно в распределение вероятностей по токенам.

Приведенное ниже описание в точности соответствует трансформатору, описанному в оригинальной статье. Существуют варианты, описанные в следующем разделе.

По традиции мы записываем все векторы в виде векторов-строк. Например, передача вектора через линейный слой означает его умножение на матрицу весов справа, как .

Токенизация

Поскольку архитектура трансформера изначально предполагает операции с числами (умножение матриц, скалярное произведение, функции активации), а не с текстом, сначала необходимо преобразовать любой входной текст в некое числовое представление. Текст очищается, разбивается на части, затем каждая часть сопоставляется с целым числом в соответствии с таблицей поиска (также называемой *словарем*). Подразделы, которых нет в словаре, обычно обозначаются специальным символом `[UNK]` ("unknown").

Словарный запас Перед токенизацией необходимо создать словарь. Этот этап называется лексификацией. Словарь можно составить на основе заданного текстового корпуса или загрузить без использования текстового корпуса.

Обычно используются такие алгоритмы, как кодирование пар байтов (BPE) и униграммная языковая модель (ULM), каждый из которых включает в себя алгоритм лексической нормализации и алгоритм токенизации. Если словарь уже известен, то нужно выполнить только токенизацию. Пример алгоритма токенизации — жадное распознавание токенов слева направо.

Embedding

Each integer token identifier is converted into an embedding vector via a lookup table. Equivalently stated, it multiplies a one-hot representation of the token identifier by an embedding matrix . For example, if the input token's identifier is , then the one-hot representation is , and its embedding vector isThe token embedding vectors are added to their respective positional encoding vectors (see below), producing the sequence of input vectors.

The dimension of an embedding vector is called *hidden size* or *embedding size* and written as .[28] This size is written as in the original transformer paper.[1]

Un-embedding

An un-embedding layer is almost the reverse of an embedding layer. Whereas an embedding layer converts a token identifier into a vector, an un-embedding layer converts a vector into a probability distribution over tokens.

The un-embedding layer is a linear-softmax layer:The matrix has shape . Some architectures use the transpose of the embedding matrix as the un-embedding matrix in order to avoid needing double the amount of embedding-related parameters and to avoid divergence during training. This practice is called *weight tying*.

Positional encoding

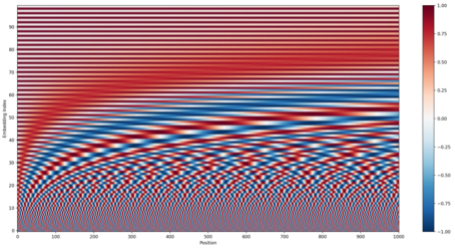


Illustration of (absolute) positional encoding with parameters

A positional encoding is a fixed-size vector representation of the relative positions of tokens within a sequence: it provides the transformer model with information about *where* the words are in the input sequence. This induces a bias towards the order of the input sequence, so that, for example, the input sequence "man bites dog" is processed differently from "dog bites man".

The positional encoding is defined as a function of type , where is a positive even integer. The full positional encoding defined in the original paper is:where .[1]

Here, is a free parameter that should be significantly larger than the biggest that would be input into the positional encoding function. The original paper uses .

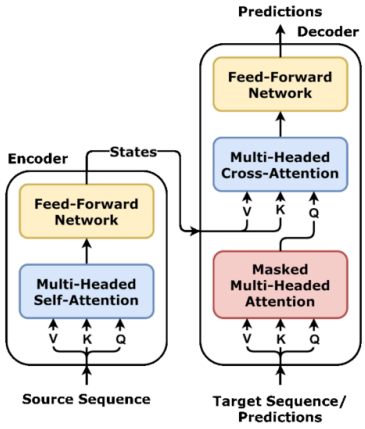
The function is in a simpler form when written as a complex function of type where .

The main reason for using this positional encoding function is that using it, shifts are linear transformations:where is the distance one wishes to shift. This allows the transformer to take any encoded position, and find the encoding of the position n-steps-ahead or n-steps-behind, by a matrix multiplication.

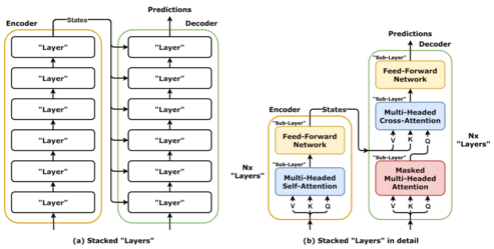
By taking a linear sum, any convolution can also be implemented as linear transformations:for any constants . This allows the transformer to take any encoded position and find a linear sum of the encoded locations of its neighbors. This sum of encoded positions, when fed into the attention mechanism, would create attention weights on its neighbors, much like what happens in a convolutional neural network language model. In the author's words, "we hypothesized it would allow the model to easily learn to attend by relative position."

In typical implementations, all operations are done over the real numbers, not the complex numbers, but since complex multiplication can be implemented as real 2-by-2 matrix multiplication, this is a mere notational difference.

Encoder–decoder (overview)



One encoder–decoder block



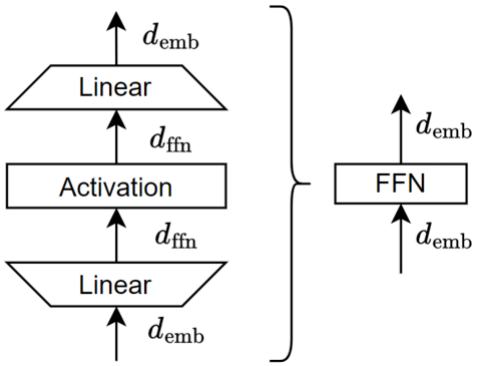
A transformer is composed of stacked encoder layers and decoder layers.

Like earlier seq2seq models, the original transformer model used an **encoder–decoder** architecture. The encoder consists of encoding layers that process all the input tokens together one layer after another, while the decoder consists of decoding layers that iteratively process the encoder's output and the decoder's output tokens so far.

The purpose of each encoder layer is to create contextualized representations of the tokens, where each representation corresponds to a token that "mixes" information from other input tokens via self-attention mechanism. Each decoder layer contains two attention sublayers: (1) cross-attention for incorporating the output of encoder (contextualized input token representations), and (2) self-attention for "mixing" information among the input tokens to the decoder (i.e. the tokens generated so far during inference time).

Both the encoder and decoder layers have a feed-forward neural network for additional processing of their outputs and contain residual connections and layer normalization steps. These feed-forward layers contain most of the parameters in a transformer model.

Feedforward network



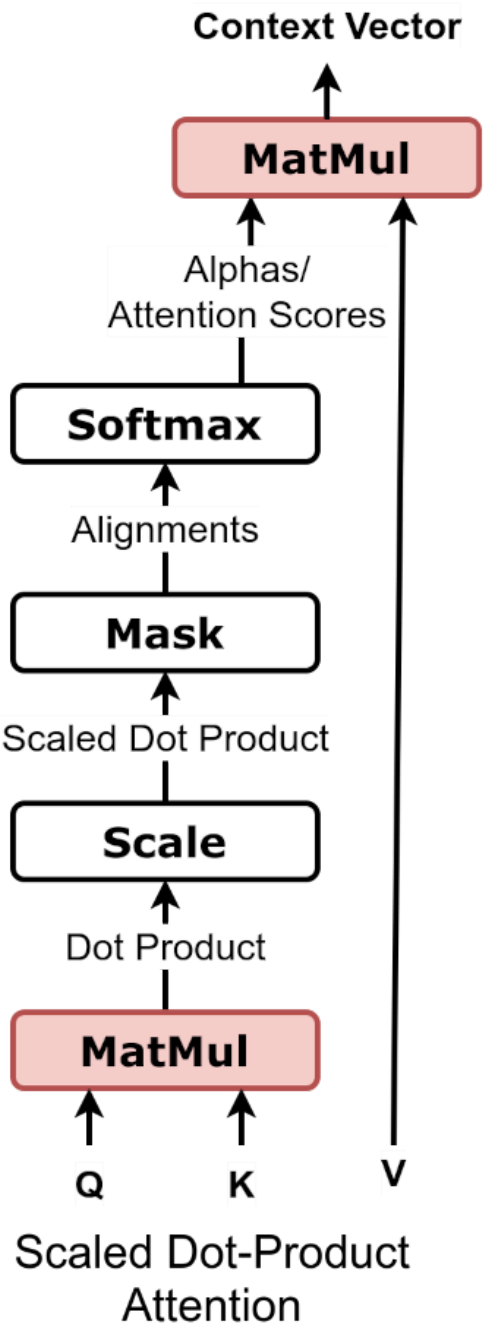
The feedforward network module. It is a two-layered network that maps -dimensional vectors into -dimensional vectors.

The feedforward network (FFN) modules in a transformer are 2-layered multilayer perceptrons:where and are weight matrices and and are bias vectors, and is its activation function. The original transformer used ReLU activation.

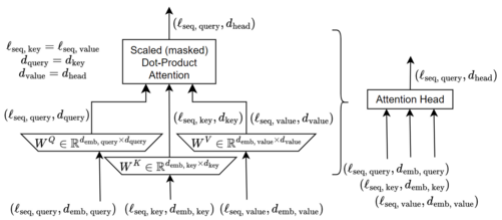
The number of neurons in the middle layer is called *intermediate size* (GPT),[44] *filter size* (BERT),[28] or *feedforward size* (BERT).[28] It is typically larger than the embedding size. For example, in both GPT-2 series and BERT series, the intermediate size of a model is 4 times its embedding size: .

Scaled dot-product attention

Attention head



Scaled dot-product attention, block diagram



Exact dimension counts within an attention head module

The attention mechanism used in the transformer architecture are scaled dot-product attention units. For each unit, the transformer model learns three weight matrices: the query weights , the key weights , and the value weights .

The module takes three sequences, a query sequence, a key sequence, and a value sequence. The query sequence is a sequence of length , and each entry is a vector of dimension . Similarly for the key and value sequences.

For each vector in the query sequence, it is multiplied by a matrix to produce a query vector . The matrix of all query vectors is the query matrix:Similarly, we construct the key matrix and the value matrix .

It is usually the case that all are square matrices, meaning , etc.

Attention weights are calculated using the query and key vectors: the attention weight from token to token is the dot product between and . The attention weights are divided by the square root of the dimension of the key vectors, , which stabilizes gradients during training, and passed through a softmax which normalizes the weights. The fact that and are different matrices allows attention to be non-symmetric: if token attends to token (i.e. is large), this does not necessarily mean that token will attend to token (i.e. could be small). The output of the attention unit for token is the weighted sum of the value vectors of all tokens, weighted by , the attention from token to each token.

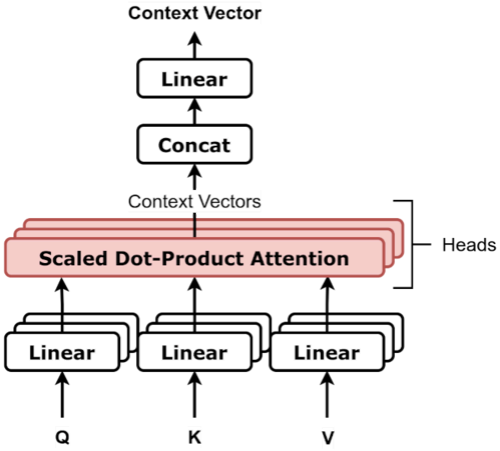
The attention calculation for all tokens can be expressed as one large matrix calculation using the softmax function, which is useful for training due to computational matrix operation optimizations that quickly compute matrix operations. The matrices , and are defined as the matrices where the th rows are vectors , , and respectively. Then we can represent the attention as

where the softmax is applied over each of the rows of the matrix.

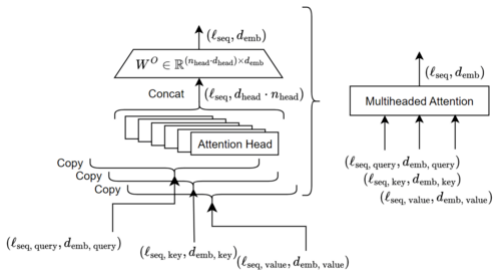
The number of dimensions in a query vector is *query size* and similarly for the *key size* and *value size* . The output dimension of an attention head is its *head dimension* . The attention mechanism requires the following three equalities to hold:but is otherwise unconstrained.

If the attention head is used in a self-attention fashion, then . If the attention head is used in a cross-attention fashion, then usually . It is theoretically possible for all three to be different, but that is rarely the case in practice.

Multihead attention



Multihead attention, block diagram



Exact dimension counts within a multihead attention module

One set of matrices is called an *attention head*, and each layer in a transformer model has multiple attention heads. While each attention head attends to the tokens that are relevant to each token, multiple attention heads allow the model to do this for different definitions of "relevance". Specifically, the query and key projection matrices, and , which are involved in the attention score computation, defines the "relevance". Meanwhile, the value projection matrix , in combination with the part of the output projection matrix , determines how the attended tokens influence what information is passed to subsequent layers and ultimately the output logits. In addition, the scope of attention, or the range of token relationships captured by each attention head, can expand as tokens pass through successive layers. This allows the model to capture more complex and long-range dependencies in deeper layers. Many transformer attention heads encode relevance relations that are meaningful to humans. For example, some attention heads can attend mostly to the next word, while others mainly attend from verbs to their direct objects.[45] The computations for each attention head can be performed in parallel, which allows for fast processing. The outputs for the attention layer are concatenated to pass into the feedforward neural network layers.

Concretely, let the multiple attention heads be indexed by , then we have where the matrix is the concatenation of word embeddings, and the matrices are "projection matrices" owned by individual attention head , and is a final projection matrix owned by the whole multihead attention module.

It is theoretically possible for each attention head to have a different head dimension , but that is rarely the case in practice.

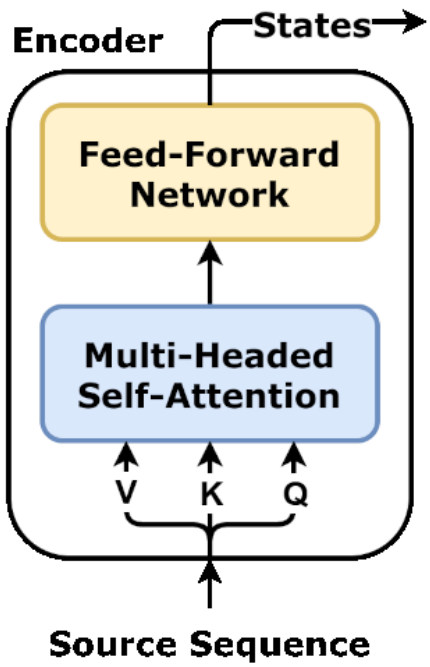
As an example, in the smallest GPT-2 model, there are only self-attention mechanisms. It has the following dimensions:Since , its output projection matrix is a square matrix.

Masked attention

The transformer architecture is constructed to calculate output tokens iteratively. Assuming refers to the calculation of the first output token , for step , the output token shall remain constant. This ensures properties of the model similar to autoregressive models.[1] Therefore, at every time step , the calculation for all outputs should not have access to tokens at position for (as it naturally is the case for time step , when tokens are not yet calculated). This behavior may be accomplished before the softmax stage by adding a mask matrix that is at entries where the attention link must be cut, and at other places: The following matrix is commonly used in decoder self-attention modules, called "causal masking":

In words, it means that each token can pay attention to itself, and every token before it, but not any after it. A non-masked attention module can be thought of as a masked attention module where the mask has all entries zero. As an example of an uncommon use of mask matrix, the XLNet considers all masks of the form , where is a random permutation matrix.[46]

Encoder



One encoder layer

An encoder consists of an embedding layer, followed by multiple encoder layers.

Each encoder layer consists of two major components: a self-attention mechanism and a feed-forward layer. It takes an input as a sequence of input vectors, applies the self-attention

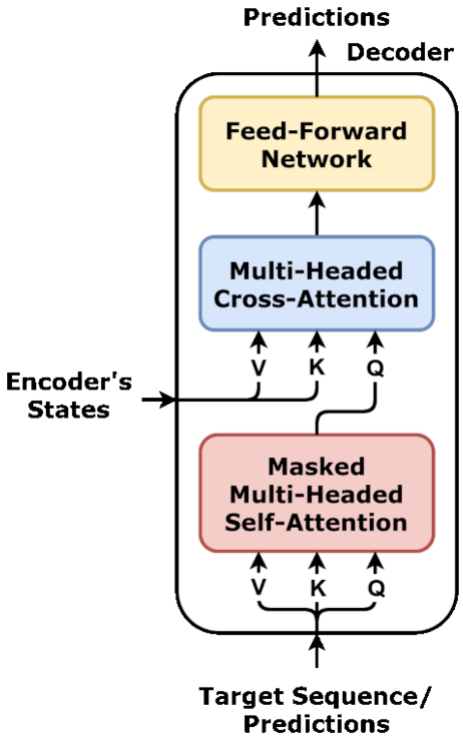
mechanism, to produce an intermediate sequence of vectors, then applies the feed-forward layer for each vector individually. Schematically, we have:

where stands for "feed-forward network". We can more succinctly write it aswith the implicit convention that the is applied to each row of the matrix individually.

The encoder layers are stacked. The first encoder layer takes the sequence of input vectors from the embedding layer, producing a sequence of vectors. This sequence of vectors is processed by the second encoder, and so on. The output from the final encoder layer is then used by the decoder.

As the encoder processes the entire input all at once, every token can attend to every other token (all-to-all attention), so there is no need for causal masking.

Decoder



One decoder layer

A decoder consists of an embedding layer, followed by multiple decoder layers, followed by an un-embedding layer.

Each decoder consists of three major components: a causally masked self-attention mechanism, a cross-attention mechanism, and a feed-forward neural network. The decoder functions in a similar fashion to the encoder, but an additional attention mechanism is inserted which instead draws relevant information from the encodings generated by the encoders. This mechanism can also be called the *encoder–decoder attention*.[1] Like the first encoder, the first decoder takes positional information and embeddings of the output sequence as its input, rather than encodings. The transformer must not use the current or future output to predict an output, so the

output sequence must be partially masked to prevent this reverse information flow.[1] This allows for autoregressive text generation. For decoding, all-to-all attention is inappropriate, because a token cannot attend to tokens not yet generated. Thus, the self-attention module in the decoder is causally masked.

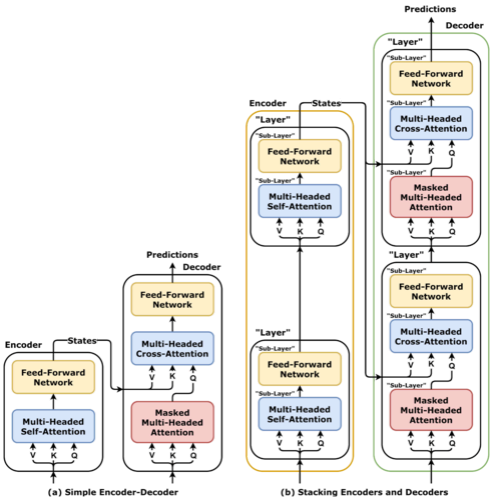
In contrast, the cross-attention mechanism attends to the output vectors of the encoder, which is computed before the decoder starts decoding. Consequently, there is no need for masking in the cross-attention mechanism.

Schematically, we have:where is the matrix with rows being the output vectors from the encoder.

The last decoder is followed by a final un-embedding layer to produce the output probabilities over the vocabulary. Then, one of the tokens is sampled according to the probability, and the decoder can be run again to produce the next token, etc., autoregressively generating output text.

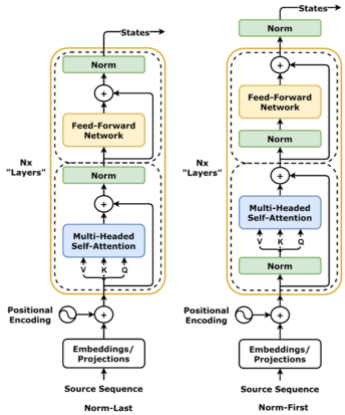
Архитектура полного трансформатора

Подслои



(а) Один слой кодировщика и один слой декодировщика. (б) Два слоя кодировщика и два слоя декодировщика. Подслои также обозначены.

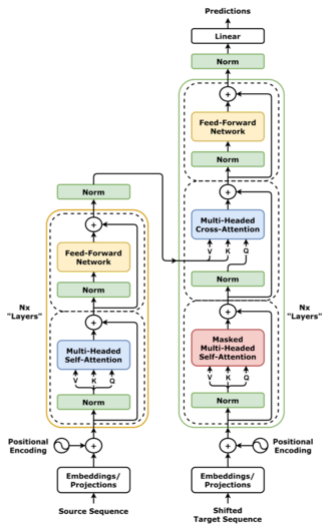
Каждый слой кодировщика содержит 2 подслоя: самовнимание и сеть прямого распространения. Каждый слой декодера содержит 3 подслоя: самовнимание с каузальной маской, перекрестное внимание и сеть прямого распространения.



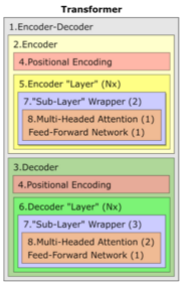
Трансформаторный энкодер с нормализацией в начале и в конце



Трансформаторный декодер с нормализацией в начале и в конце



Блок-схема полной архитектуры трансформатора



Schematic object hierarchy for the full transformer architecture, in object-oriented programming style

The final points of detail are the residual connections and layer normalization, (denoted as "LayerNorm", or "LN" in the following), which while conceptually unnecessary, are necessary for numerical stability and convergence.

The residual connections are introduced to avoid vanishing gradient issues and stabilize the training process. They can be expressed by , where is a given component of the transformer. Adding the input can preserve the input information and avoid issues when the gradient of is close to zero.

Similarly to how the feedforward network modules are applied individually to each vector, the LayerNorm is also applied individually to each vector.

There are two common conventions in use: the *post-LN* and the *pre-LN* convention. In the post-LN convention, the output of each sublayer is where is the function implemented by the sublayer itself.

In the pre-LN convention, the output of each sublayer isThe original 2017 transformer used the post-LN convention. It was difficult to train and required careful hyperparameter tuning and a "warm-up" in learning rate, where it starts small and gradually increases. The pre-LN convention, proposed several times in 2018,[47] was found to be easier to train, requiring no warm-up, leading to faster convergence.[36]

Pseudocode

The following is the pseudocode for a standard pre-LN encoder–decoder transformer, adapted from *Formal Algorithms for Transformers*.[48]

```
input: Encoder input t_e
       Decoder input t_d
output: Array of probability distributions, with shape (decoder
vocabulary size x length(decoder output sequence))

/* encoder */
```

```
z_e ← encoder.tokenizer(t_e)

for each t in 1:length(z_e) do
    z_e[t] ← encoder.embedding(z_e[t]) +
encoder.positional_embedding(t)

for each l in 1:length(encoder.layers) do
    layer ← encoder.layers[l]

    /* first sublayer */
    z_e_copy ← copy(z_e)
    for each t in 1:length(z_e) do
        z_e[t] ← layer.layer_norm(z_e[t])
    z_e ← layer.multihead_attention(z_e, z_e, z_e)
    for each t in 1:length(z_e) do
        z_e[t] ← z_e[t] + z_e_copy[t]

    /* second sublayer */
    z_e_copy ← copy(z_e)
    for each t in 1:length(z_e) do
        z_e[t] ← layer.layer_norm(z_e[t])
    z_e ← layer.feedforward(z_e)
    for each t in 1:length(z_e) do
        z_e[t] ← z_e[t] + z_e_copy[t]

for each t in 1:length(z_e) do
    z_e[t] ← encoder.final_layer_norm(z_e[t])

/* decoder */
z_d ← decoder.tokenizer(t_d)

for each t in 1:length(z_d) do
    z_d[t] ← decoder.embedding(z_d[t]) +
decoder.positional_embedding(t)

for each l in 1:length(decoder.layers) do
        layer ← decoder.layers[l]

        /* first sublayer */
```

```
        z_d_copy ← copy(z_d)
        for each t in 1:length(z_d) do
            z_d[t] ← layer.layer_norm(z_d[t])
        z_d ← layer.masked_multihead_attention(z_d, z_d, z_d)
        for each t in 1:length(z_d) do
            z_d[t] ← z_d[t] + z_d_copy[t]

        /* second sublayer */
        z_d_copy ← copy(z_d)
        for each t in 1:length(z_d) do
            z_d[t] ← layer.layer_norm(z_d[t])
        z_d ← layer.multihead_attention(z_d, z_e, z_e)
       for each t in 1:length(z_d) do
           z_d[t] ← z_d[t] + z_d_copy[t]

        /* third sublayer */
        z_d_copy ← copy(z_d)
        for each t in 1:length(z_d) do
            z_d[t] ← layer.layer_norm(z_d[t])
        z_d ← layer.feedforward(z_d)
        for each t in 1:length(z_d) do
            z_d[t] ← z_d[t] + z_d_copy[t]

z_d ← decoder.final_layer_norm(z_d)

output_distributions ← []
for each t in 1:length(z_d) do
    output_distributions.append(decoder.unembed(z_d[t]))

return output_distributions
```

Terminology

The transformer architecture, being modular, allows variations. Several common variations are described here.[39]

An "encoder-only" transformer applies the encoder to map an input text into a sequence of vectors that represent the input text. This is usually used for text embedding and representation learning for downstream applications. BERT is encoder-only. They are less often used currently,

as they were found to be not significantly better than training an encoder–decoder transformer, then taking just the encoder.[43] They are also referred to as "all-to-all" or "BERT-like".

A "decoder-only" transformer is not literally decoder-only, since without an encoder, the cross-attention mechanism has nothing to attend to. Thus, the decoder layers in a decoder-only transformer are composed of just two sublayers: the causally masked self-attention, and the feedforward network. This is usually used for text generation and instruction following. The models in the GPT series and Chinchilla series are decoder-only. They are also referred to as "autoregressive" or "causal".

An "encoder–decoder" transformer is generally the same as the original transformer, with 2 sublayers per encoder layer and 3 sublayers per decoder layer, etc. They might have minor architectural improvements, such as alternative activation functions, changing the location of normalization, etc. This is also usually used for text generation and instruction following. The models in the T5 series are encoder–decoder.[39]

A "prefixLM" (prefix language model) is a decoder-only architecture, but with prefix masking, which is different from causal masking. Specifically, it has mask of the form[39]:Figure 3[49] where the first columns correspond to the "prefix", and the subsequent columns correspond to the autoregressively generated text based on the prefix. They resemble encoder–decoder models, but has less "sparsity". Such models are rarely used, though they are cited as theoretical possibilities and benchmarked comparisons.[43]

There are also mixed seq2seq models. For example, in 2020, Google Translate replaced the previous RNN-encoder–RNN-decoder model with a transformer-encoder–RNN-decoder model, as transformer-based decoders did not appear to significantly increase quality unlike the encoder, while the RNN decoder was much faster.[30]

Последующие работы

В архитектуру Transformer было внесено множество улучшений. Стандартная улучшенная конфигурация — Transformer++, в которой используются RoPE, RMSNorm и модуль прямой передачи SwiGLU.[50]

Альтернативный модуль прямой связи

Исходный модуль прямой передачи данных в трансформерах представляет собой стандартную линейную-ReLU-линейную сеть. Были разработаны и другие модули прямой передачи данных. В серии Llama и PaLM используется SwiGLU;[51] в GPT-1 и BERT[28] используется GELU.[52]

Альтернативные функции активации часто используются в сочетании с Gated Linear Units в модуле прямой передачи. [51]

Альтернативные нормализации

Нормализация, используемая в трансформерах, может отличаться от LayerNorm. Один из примеров — RMSNorm[53] , который используется в серии Llama. Другие примеры — ScaleNorm[54] и FixNorm.[54]

Альтернативные позиционные кодировки

Трансформеры могут использовать и другие методы позиционного кодирования, помимо синусоидального.[55]

В оригинальной статье о трансформерах сообщалось об использовании обученного позиционного кодирования,[56] но было установлено, что оно не превосходит синусоидальное.[1] Позже,[57] было обнаружено, что причинно-следственная маскировка сама по себе обеспечивает достаточный сигнал для декодера трансформера, чтобы он мог научиться неявно выполнять абсолютное позиционное кодирование без модуля позиционного кодирования.

Эффективная реализация

Трансформер-модель была реализована в стандартных фреймворках для глубокого обучения, таких как TensorFlow и PyTorch. *Transformers* — это библиотека, созданная Hugging Face, которая предоставляет архитектуры на основе трансформеров и предварительно обученные модели. [9]

KV-кэширование

Когда авторегрессионный трансформер используется для логического вывода, например для генерации текста, вектор запроса на каждом шаге разный, но уже вычисленные векторы ключа и значения всегда одни и те же. Метод **кэширования KV** сохраняет вычисленные векторы ключа и значения в каждом блоке внимания, чтобы они не вычислялись заново при появлении каждого нового токена. PagedAttention применяет страничную организацию памяти к кэшированию KV. [58][59][60]

If a transformer is used with a baked-in prompt, such as ["You are a customer support agent..."], then the key and value vectors can be computed for the prompt, and saved on disk. The saving in compute is significant when the model is used for many short real-time interactions, such as in online chatbots.

In general, when a user uses an autoregressive transformer to generate a continuation to a sequence of tokens, the model would first perform a forward-pass on this sequence, whereby the KV caches over this sequence are computed. This is called **prefilling**. After prefilling, the actual

token generation is called **decoding**. Hyperscalers serving large Transformer models may use **disaggregated inference**, wherein prefilling and decoding are performed on separately specialized hardware.[61]

Continuous Batching

When using autoregressive transformer models, requests can have different input lengths and can finish generating at different times. Continuous batching adds new requests to batch and removes completed requests between decoding iterations instead of waiting for every request in a batch to finish. This improves hardware utilization and can reduce latency and increase throughput for systems that serve many concurrent requests.[62]

Quantization

Transformer models are often quantized for inference, meaning that weights, activations, or both are represented with lower-precision numerical formats than the floating point formats used during training. Quantization can reduce memory use and data transfer costs and can improve inference speed on hardware that supports low precision arithmetic. The trade off is that aggressive quantization can come at the cost of model accuracy or may need calibration and specialized quantization methods to perform well.[63]

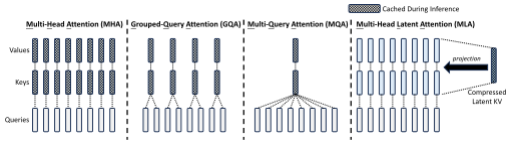
FlashAttention

FlashAttention[64] is an algorithm that implements the transformer attention mechanism efficiently on a GPU. It is a communication-avoiding algorithm that performs matrix multiplications in blocks, such that each block fits within the cache of a GPU, and by careful management of the blocks it minimizes data copying between GPU caches (as data movement is slow).

The FlashAttention method is a communication-avoiding algorithm that fuses these operations into a single loop, increasing the arithmetic intensity. It is an online algorithm that computes the following quantities:[65][64]and returns . In practice, FlashAttention operates over multiple queries and keys per loop iteration, in a similar way as blocked matrix multiplication. If backpropagation is needed, then the output vectors and the intermediate arrays are cached, and during the backward pass, attention matrices are rematerialized from these, making it a form of gradient checkpointing.

An improved version, FlashAttention-2, was developed to handle longer context lengths.[66]

Multi-Query Attention



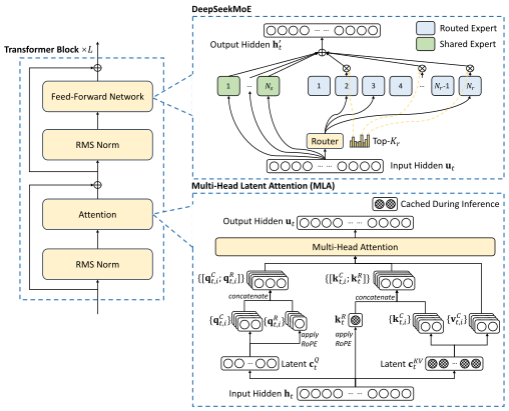
Comparison between several different forms of attention mechanism and the amount of KV caching necessary for each

Multi-Query Attention (MQA) changes the Multihead Attention mechanism.[67] Whereas normally,

with Multi-Query Attention, there is just one , thus:

This has a neutral effect on model quality and training speed, but increases inference speed.

More generally, grouped-query attention (GQA) partitions attention heads into groups, each of which shares the key-value pair. MQA is GQA with one group, while standard Multihead Attention is GQA with the maximal number of groups.[68]



The architecture of V2, showing both MLA and a variant of mixture of experts[69]:Figure 2

Multihead Latent Attention (MLA) is a low-rank approximation to MQA. Specifically, each hidden vector, before entering the attention mechanism, is first projected to two low-dimensional spaces ("latent space"), one for query and one for key-value (KV vector). This design minimizes the KV cache, as only the low-dimensional KV vector needs to be cached.[69]

Speculative decoding

Speculative decoding[70][71] is a method to accelerate token decoding. Similarly to speculative execution in CPUs, future tokens are computed quickly, then verified. If the quickly computed tokens are incorrect, they are discarded and computed slowly.

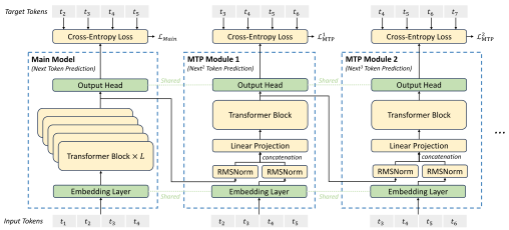
The key factor in speculative decoding is that a transformer decoder can verify faster than it can decode, in the following sense.

Suppose we have two transformer models like GPT-3 and GPT-3-small, both with a context window size of 512. To generate an entire context window autoregressively with greedy decoding with GPT-3, it must be run for 512 times, each time generating a token , taking time . However, if

we had some educated guess for the values of these tokens, we could verify all of them in parallel, in one run of the model, by checking that each is indeed the token with the largest log-likelihood in the -th output.

In speculative decoding, a smaller model or some other simple heuristic is used to generate a few speculative tokens that are subsequently verified by the larger model. For example, suppose we use GPT-3-small to generate four speculative tokens: . This only takes . These tokens are then run through the larger GPT-3 in one go. Suppose that and are verified by GPT-3 as what it would have picked, then those are kept, but is not, so are discarded, and GPT-3 is run on those. This would take , which might be shorter than .

For non-greedy decoding, similar ideas apply, except the speculative tokens are accepted or rejected stochastically, in a way that guarantees the final output distribution is the same as if speculative decoding was not used.[70][72]



Multi-token prediction

In Multi-Token Prediction, a single forward pass creates a final embedding vector, which then is un-embedded into a token probability. However, that vector can then be further processed by another transformer block to predict the *next* token, and so on for arbitrarily many steps into the future. This trades off accuracy for speed, since each new token costs just one more transformer block, rather than the entire stack.[73][74]

Sub-quadratic transformers

Training transformer-based architectures can be expensive, especially for long inputs.[75] Many methods have been developed to attempt to address the issue. In the image domain, Swin transformer is an efficient architecture that performs attention inside shifting windows.[76] In the audio domain, SepTr decouples the attention in time and frequency domains.[77] *Long Range Arena* (2020)[78] is a standard benchmark for comparing the behavior of transformer architectures over long inputs.

The standard attention graph is either all-to-all or causal, both of which scales as where is the number of tokens in a sequence. In both cases, there are pairs of tokens such that one influences the other within the attention block. By pruning the attention graph, the number of pairs of token influences can drop below . Such architectures are often referred to as *sparse* attention, or sparse Transformers.

Sparse attention[79] uses attention graphs that grows slower than . For example, BigBird (2020)[80] uses random small-world networks which grows as .

Multimodality

Transformers can also be used/adapted for modalities (input or output) beyond just text, usually by finding a way to "tokenize" the modality.

Multimodal models can either be trained from scratch, or by finetuning. A 2022 study found that transformers pretrained only on natural language can be finetuned on only 0.03% of parameters and become competitive with LSTMs on a variety of logical and visual tasks, demonstrating transfer learning.[81] The LLaVA was a vision-language model composed of a language model (Vicuna-13B)[82] and a vision model (ViT-L/14), connected by a linear layer. Only the linear layer is finetuned.[83]

Vision transformers adapt the transformer to computer vision by breaking down input images as a series of patches, turning them into vectors, and treating them like embedding vectors of tokens in a standard transformer.

Conformer and later Whisper[84] follow the same pattern for speech recognition, first turning the speech signal into a spectrogram, which is then treated like an image, i.e. broken down into a series of patches, turned into vectors and treated like embedding vector of tokens in a standard transformer.

Perceivers[85][86] are a variant of transformers designed for multimodality.

For image generation, notable architectures are DALL-E 1 (2021), Parti (2022),[87] Phenaki (2023),[88] and Muse (2023).[89] Unlike later models, DALL-E is not a diffusion model. Instead, it uses a decoder-only transformer that autoregressively generates a text, followed by the token representation of an image, which is then converted by a variational autoencoder to an image.[90] Parti is an encoder–decoder transformer, where the encoder processes a text prompt, and the decoder generates a token representation of an image.[91] Muse is an encoder-only transformer that is trained to predict masked image tokens from unmasked image tokens. During generation, all input tokens are masked, and the highest-confidence predictions are included for the next iteration, until all tokens are predicted.[89] Phenaki is a text-to-video model. It is a bidirectional masked transformer conditioned on pre-computed text tokens. The generated tokens are then decoded to a video.[88]

Приложения

Трансформеры добились больших успехов в обработке естественного языка (NLP). Многие большие языковые модели, такие как GPT-2, GPT-3, GPT-4, Gemini, AlbertAGPT,

Claude, BERT, Grok, XLNet, RoBERTa и ChatGPT, демонстрируют способность трансформеров выполнять широкий спектр подзадач, связанных с NLP, и их применение в реальных условиях, в том числе:

- машинный перевод
- прогнозирование временных рядов
- обобщение документов
- генерация документов
- распознавание именованных сущностей (NER)[92]
- написание компьютерного кода на основе требований, выраженных на естественном языке.
- преобразование речи в текст

Помимо традиционного НЛП, архитектура трансформера успешно применяется в других областях, таких как:

- анализ биологических последовательностей
- понимание видео
- сворачивание белков (например, альфа-сворачивание)
- оцениваю позиции на шахматной доске. Используя только статическую оценку (то есть без минимаксного поиска), transformer добился Elo, равного 2895, что соответствует уровню гроссмейстера.[8]
- встраивание цепочки сервисных функций[93]

См. также

- seq2seq – Семейство подходов к машинному обучению
- Circuit (нейронная сеть) – Интерпретируемые вычислительные подграфы в искусственных нейронных сетях
- Perceiver – Вариант Transformer, разработанный для мультимодальных данных
- Vision transformer – Модель машинного обучения для обработки изображений
- Large language model – Тип модели машинного обучения
- BERT (языковая модель) – Серия языковых моделей, разработанных Google AI
- Генеративный предварительно обученный трансформер – Тип большой языковой модели
- T5 (языковая модель) – серия больших языковых моделей, разработанных Google AI

Примечания

1. Управляемые рекуррентные блоки (2014) еще больше снизили сложность алгоритма.
2. Некоторые архитектуры, такие как RWKV (Receptance Weighted Key Value) или модели пространства состояний, позволяют избежать этой проблемы.

Ссылки

1. Васвани, Ашиш; Шазир, Ноам; Пармар, Ники; Ушкорайт, Якоб; Джонс, Лион; Гомес, Эйдан Н.; Кайзер, Лукаш; Полосухин, Илья (2017). «Внимание — это все, что вам нужно» (https://web.archive.org/web/20240221141113/https://proceedings.neurips.cc/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf) ((PDF)). *31-я конференция по нейронным системам обработки информации (NIPS 2017), 4-9 декабря 2017 года, Лонг-Бич, Калифорния, США*. Том. 30. Curran Associates, Inc. arXiv:1706.03762 (https://arxiv.org/abs/1706.03762) . ISBN 978-1-5108-6096-4. Архивировано из оригинала (https://proceedings.neurips.cc/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf) 21 февраля 2024 года. Дата обращения: 31 октября 2023.
2. Хохрайтер, Зепп; Шмидхубер, Юрген (ноябрь 1997). "Долгая краткосрочная память". *Нейронные вычисления*. **9** (8): 1735–1780. doi:10.1162/neco.1997.9.8.1735 (https://doi.org/10.1162%2Fneco.1997.9.8.1735) . PMID 9377276 (https://pubmed.ncbi.nlm.nih.gov/9377276) .
3. Богданов; Чо, Кёнхюн; Бенжио, Йошуа (1 сентября 2014 г.). «Нейронный машинный перевод путем совместного обучения выравниванию и переводу». arXiv:1409.0473 (https://arxiv.org/abs/1409.0473) [cs.CL (https://arxiv.org/archive/cs.CL)].
4. Луонг, Минь-Тханг; Фам, Хьеу; Мэннинг, Кристофер Д. (17 августа 2015 г.). «Эффективные подходы к нейронному машинному переводу на основе внимания». arXiv:1508.04025 (https://arxiv.org/abs/1508.04025) [cs.CL (https://arxiv.org/archive/cs.CL)].
5. Паризотто, Эмилио; Сонг, Фрэнсис; Рэй, Джек; Паскану, Разван; Гюльджере, Чаглар; Джаякумар, Сиддхант; Ядерберг, Макс; Кауфман, Рафаэль Лопес; Кларк, Эйдан; Нури, Себ; Ботвиник, Мэтью; Хис, Николя; Хадселл, Райя (21 ноября 2020 г.). «Стабилизация трансформеров для обучения с подкреплением» (https://proceedings.mlr.press/v119/parisotto20a.html) . *Материалы 37-й Международной конференции по машинному обучению*. PMLR: 7487–7498. Архивировано (https://web.archive.org/web/20240809053144/https://proceedings.mlr.press/v119/parisotto20a.html) из оригинала 09.08.2024. Дата обращения: 09.08.2024.

6. Рэдфорд, Алек; Ким, Джонг Вук; Сюй, Тао; Брокман, Грег; МакЛиви, Кристин; Суцкевер, Илья (2022). "Надежное распознавание речи с помощью крупномасштабного слабого контроля". arXiv:2212.04356 (https://arxiv.org/abs/2212.04356) [eess.AS (https://arxiv.org/archive/eess.AS)].
7. Монастырский, Максим; Азулай, Ошер; Синтов, Авишай (февраль 2023 г.). "Обучение метанию с помощью нескольких образцов с использованием решающих трансформеров". *IEEE Robotics and Automation Letters*. **8** (2): 576–583. Bibcode:2023IRAL....8..576M (https://scixplorer.org/abs/2023IRAL....8..576M) . doi:10.1109/LRA.2022.3229266 (https://doi.org/10.1109%2FLRA.2022.3229266) .
8. Руосс, Аниан; Делетанг, Грегуар; Медапати, Сураб; Грау-Мойя, Жорди; Ли, Вэньлян; Кэтт, Эллиот; Рид, Джон; Геневейн, Тим (7 февраля 2024 г.). «Шахматы на уровне гроссмейстера без поиска». arXiv:2402.04494v1 (https://arxiv.org/abs/2402.04494v1) [cs.LG (https://arxiv.org/archive/cs.LG)].
9. Wolf, Thomas; Debut, Lysandre; Sanh, Victor; Chaumond, Julien; Delangue, Clement; Moi, Anthony; Cistac, Pierric; Rault, Tim; Louf, Remi; Funtowicz, Morgan; Davison, Joe; Shleifer, Sam; von Platen, Patrick; Ma, Clara; Jernite, Yacine; Plu, Julien; Xu, Canwen; Le Scao, Teven; Gugger, Sylvain; Drame, Mariama; Lhoest, Quentin; Rush, Alexander (2020). "Transformers: State-of-the-Art Natural Language Processing". *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. pp. 38–45. doi:10.18653/v1/2020.emnlp-demos.6 (https://doi.org/10.18653%2Fv1%2F2020.emnlp-demos.6) .
10. Feldman, J; Ballard, D (September 1982). "Connectionist models and their properties". *Cognitive Science*. **6** (3): 205–254. doi:10.1016/S0364-0213(82)80001-3 (https://doi.org/10.1016%2FS0364-0213%2882%2980001-3) .
11. Rumelhart, David E.; Hinton, Geoffrey E.; McClelland, James L. (1986). "Interactive Processes in Speech Perception". *Parallel Distributed Processing*. doi:10.7551/mitpress/5236.003.0005 (https://doi.org/10.7551%2Fmitpress%2F5236.003.0005) . ISBN 978-0-262-29140-8.
12. Giles, C. Lee; Maxwell, Tom (December 1987). "Learning, invariance, and generalization in high-order neural networks". *Applied Optics*. **26** (23): 4972–4978. doi:10.1364/AO.26.004972 (https://doi.org/10.1364%2FAO.26.004972) . PMID 20523475 (https://pubmed.ncbi.nlm.nih.gov/20523475) .

13. Cho, Kyunghyun; van Merriënboer, Bart; Gulcehre, Caglar; Bahdanau, Dzmitry; Bougares, Fethi; Schwenk, Holger; Bengio, Yoshua (October 2014). "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation" (https://aclanthology.org/D14-1179) . In Moschitti, Alessandro; Pang, Bo; Daelemans, Walter (eds.). *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Doha, Qatar: Association for Computational Linguistics. pp. 1724–1734. arXiv:1406.1078 (https://arxiv.org/abs/1406.1078) . doi:10.3115/v1/D14-1179 (https://doi.org/10.3115%2Fv1%2FD14-1179) . Archived (https://web.archive.org/web/20240906214240/https://aclanthology.org/D14-1179/) from the original on 2024-09-06. Retrieved 2024-08-09.

14. Sutskever, Ilya; Vinyals, Oriol; Le, Quoc Viet (14 Dec 2014). "Sequence to sequence learning with neural networks". arXiv:1409.3215 (https://arxiv.org/abs/1409.3215) [cs.CL (https://arxiv.org/archive/cs.CL)]. [first version posted to arXiv on 10 Sep 2014]

15. Chung, Junyoung; Gulcehre, Caglar; Cho, KyungHyun; Bengio, Yoshua (2014). "Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling". arXiv:1412.3555 (https://arxiv.org/abs/1412.3555) [cs.NE (https://arxiv.org/archive/cs.NE)].

16. Gruber, Nicole; Jockisch, Alfred (30 June 2020). "Are GRU Cells More Specific and LSTM Cells More Sensitive in Motive Classification of Text?" (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC7861254) . *Frontiers in Artificial Intelligence*. **3** 40. doi:10.3389/frai.2020.00040 (https://doi.org/10.3389%2Ffrai.2020.00040) . PMC 7861254 (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC7861254) . PMID 33733157 (https://pubmed.ncbi.nlm.nih.gov/33733157) .

17. Sutskever, Ilya; Vinyals, Oriol; Le, Quoc V (2014). "Sequence to Sequence Learning with Neural Networks" (https://web.archive.org/web/20250127094117/https://proceedings.neurips.cc/paper/2014/hash/a14ac55a4f27472c5d894ec1c3c743d2-Abstract.html) . *Advances in Neural Information Processing Systems*. **27**. Curran Associates, Inc. arXiv:1409.3215 (https://arxiv.org/abs/1409.3215) . Archived from the original (https://proceedings.neurips.cc/paper/2014/hash/a14ac55a4f27472c5d894ec1c3c743d2-Abstract.html) on 2025-01-27. Retrieved 2024-07-23.

18. Luong, Minh-Thang; Pham, Hieu; Manning, Christopher D. (2015). "Effective Approaches to Attention-based Neural Machine Translation". arXiv:1508.04025 (https://arxiv.org/abs/1508.04025) [cs.CL (https://arxiv.org/archive/cs.CL)].

19. Wu, Yonghui; et al. (2016-09-01). "Google's Neural Machine Translation System: Bridging the Gap between Human and Machine Translation". arXiv:1609.08144 (https://arxiv.org/abs/1609.08144) [cs.CL (https://arxiv.org/archive/cs.CL)].

20. Lewis-Kraus, Gideon (2016-12-14). "The Great A.I. Awakening" (https://web.archive.org/web/20230524052626/https://www.nytimes.com/2016/12/14/magazine/the-great-ai-awakening.html) . *The New York Times*. Archived from the original (https://www.nytimes.com/2016/12/14/magazine/the-great-ai-awakening.html) on 24 May 2023. Retrieved 2023-06-22.

21. Parikh, Ankur P.; Täckström, Oscar; Das, Dipanjan; Uszkoreit, Jakob (2016-09-25). "A Decomposable Attention Model for Natural Language Inference". arXiv:1606.01933 (https://arxiv.org/abs/1606.01933) [cs.CL (https://arxiv.org/archive/cs.CL)].

22. Levy, Steven. "8 Google Employees Invented Modern AI. Here's the Inside Story" (https://www.wired.com/story/eight-google-employees-invented-modern-ai-transformers-paper/) . *Wired*. Archived (https://web.archive.org/web/20240320101528/https://www.wired.com/story/eight-google-employees-invented-modern-ai-transformers-paper/) from the original on 20 Mar 2024. Retrieved 2024-08-06.

23. Cheng, Jianpeng; Dong, Li; Lapata, Mirella (November 2016). "Long Short-Term Memory-Networks for Machine Reading" (https://aclanthology.org/D16-1053/) . In Su, Jian; Duh, Kevin; Carreras, Xavier (eds.). *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*. Austin, Texas: Association for Computational Linguistics. pp. 551–561. doi:10.18653/v1/D16-1053 (https://doi.org/10.18653%2Fv1%2FD16-1053) . Archived (https://web.archive.org/web/20240827143929/https://aclanthology.org/D16-1053/) from the original on 2024-08-27. Retrieved 2024-08-27.

24. Peng, Bo; Alcaide, Eric; Anthony, Quentin; Albalak, Alon; Arcadinho, Samuel; Biderman, Stella; Cao, Huanqi; Cheng, Xin; Chung, Michael (2023-12-10). "RWKV: Reinventing RNNs for the transformer Era". arXiv:2305.13048 (https://arxiv.org/abs/2305.13048) [cs.CL (https://arxiv.org/archive/cs.CL)].

25. Marche, Stephen (2024-08-23). "Was Linguistic A.I. Created by Accident?" (https://www.newyorker.com/science/annals-of-artificial-intelligence/was-linguistic-ai-created-by-accident) . *The New Yorker*. Retrieved 2024-08-27.

26. Vaswani, Ashish; Bengio, Samy; Brevdo, Eugene; Chollet, Francois; Gomez, Aidan; Gouws, Stephan; Jones, Llion; Kaiser, Łukasz; Kalchbrenner, Nal; Parmar, Niki; Sepassi, Ryan; Shazeer, Noam; Uszkoreit, Jakob (March 2018). Cherry, Colin; Neubig, Graham (eds.). "Tensor2Tensor for Neural Machine Translation" (https://aclanthology.org/W18-1819/) . *Proceedings of the 13th Conference of the Association for Machine Translation in the Americas (Volume 1: Research Track)*. Boston, MA: Association for Machine Translation in the Americas: 193–199. Archived (https://web.archive.org/web/20260429230143/https://aclanthology.org/W18-1819/) from the original on 2026-04-29. Retrieved 2026-03-31.

27. Kaiser, Łukasz (2017-06-19). "Accelerating Deep Learning Research with the Tensor2Tensor Library" (https://research.google/blog/accelerating-deep-learning-research-with-the-tensor2tensor-library/) . *Google Research Blog*. Archived (https://web.archive.org/web/20260415200010/https://research.google/blog/accelerating-deep-learning-research-with-the-tensor2tensor-library/) from the original on 2026-04-15. Retrieved 2026-03-31.

28. Devlin, Jacob; Chang, Ming-Wei; Lee, Kenton; Toutanova, Kristina (11 October 2018). "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding". arXiv:1810.04805v2 (https://arxiv.org/abs/1810.04805v2) [cs.CL (https://arxiv.org/archive/cs.CL)].

29. "Google: BERT now used on almost every English query" (https://searchengineland.com/google-bert-used-on-almost-every-english-query-342193) . *Search Engine Land*. 2020-10-15. Archived (https://web.archive.org/web/20220506220519/https://searchengineland.com/google-bert-used-on-almost-every-english-query-342193) from the original on 2022-05-06. Retrieved 2020-11-24.

30. Caswell, Isaac; Liang, Bowen (June 8, 2020). "Recent Advances in Google Translate" (https://research.google/blog/recent-advances-in-google-translate/) . *Google Research*. Archived (https://web.archive.org/web/20240704042433/https://research.google/blog/recent-advances-in-google-translate/) from the original on 4 Jul 2024. Retrieved 2024-08-07.

31. "The inside story of how ChatGPT was built from the people who made it" (https://www.technologyreview.com/2023/03/03/1069311/inside-story-oral-history-how-chatgpt-built-openai/) . *MIT Technology Review*. Archived (https://web.archive.org/web/20230303093219/https://www.technologyreview.com/2023/03/03/1069311/inside-story-oral-history-how-chatgpt-built-openai/) from the original on 2023-03-03. Retrieved 2024-08-06.

32. "Introducing ChatGPT" (https://openai.com/index/chatgpt/) . *OpenAI*. 2022-11-30. Archived (https://web.archive.org/web/20260405004654/https://openai.com/index/chatgpt/) from the original on 2026-04-05. Retrieved 2026-05-16.

33. Deign, Jason (December 6, 2024). "How ChatGPT changed… well, almost everything" (https://newsroom.cisco.com/c/r/newsroom/en/us/a/y2024/m12/how-chatgpt-changed-well-almost-everything.html) . *Cisco Newsroom*. Retrieved 2026-10-22.

34. Kaiser, Lukasz; Gomez, Aidan N.; Shazeer, Noam; Vaswani, Ashish; Parmar, Niki; Jones, Llion; Uszkoreit, Jakob (2017-06-16). "One Model To Learn Them All". arXiv:1706.05137v1 (https://arxiv.org/abs/1706.05137v1) [cs.LG (https://arxiv.org/archive/cs.LG)].

35. Liu, Zhuang; Mao, Hanzi; Wu, Chao-Yuan; Feichtenhofer, Christoph; Darrell, Trevor; Xie, Saining (2022). *A ConvNet for the 2020s* (https://openaccess.thecvf.com/content/CVPR2022/html/Liu_A_ConvNet_for_the_2020s_CVPR_2022_paper.html) . Conference on Computer Vision and Pattern Recognition (CVPR). pp. 11976–11986.

36. Xiong, Ruibin; Yang, Yunchang; He, Di; Zheng, Kai; Zheng, Shuxin; Xing, Chen; Zhang, Huishuai; Lan, Yanyan; Wang, Liwei; Liu, Tie-Yan (2020-06-29). "On Layer Normalization in the Transformer Architecture". arXiv:2002.04745 (https://arxiv.org/abs/2002.04745) [cs.LG (https://arxiv.org/archive/cs.LG)].

37. Vaswani, Ashish; Shazeer, Noam; Parmar, Niki; Uszkoreit, Jakob; Jones, Llion; Gomez, Aidan N; Kaiser, Ł ukasz; Polosukhin, Illia (2017). "Attention is All you Need" (https://papers.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html) . *Advances in Neural Information Processing Systems*. **30**. Curran Associates, Inc.

38. Narayanan, Deepak; Harlap, Aaron; Phanishayee, Amar; Seshadri, Vivek; Devanur, Nikhil R.; Ganger, Gregory R.; Gibbons, Phillip B.; Zaharia, Matei (2019). "PipeDream: Generalized pipeline parallelism for DNN training". *Proceedings of the 27th ACM Symposium on Operating Systems Principles*. pp. 1–15. doi:10.1145/3341301.3359646 (https://doi.org/10.1145%2F3341301.3359646) . ISBN 978-1-4503-6873-5.

39. Raffel, Colin; Shazeer, Noam; Roberts, Adam; Lee, Katherine; Narang, Sharan; Matena, Michael; Zhou, Yanqi; Li, Wei; Liu, Peter J. (2020). "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer" (https://www.jmlr.org/papers/v21/20-074.html) . *Journal of Machine Learning Research*. **21** (140): 1–67. Archived (https://web.archive.org/web/20260410072046/https://jmlr.org/papers/v21/20-074.html) from the original on 2026-04-10. Retrieved 2026-04-18.

40. Raffel, Colin; Shazeer, Noam; Roberts, Adam; Lee, Katherine; Narang, Sharan; Matena, Michael; Zhou, Yanqi; Li, Wei; Liu, Peter J. (2019). "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer". arXiv:1910.10683 (https://arxiv.org/abs/1910.10683) [cs.LG (https://arxiv.org/archive/cs.LG)].

41. "Masked language modeling" (https://huggingface.co/docs/transformers/tasks/masked_language_modeling) . *huggingface.co*. Archived (https://web.archive.org/web/20231010083915/https://huggingface.co/docs/transformers/tasks/masked_language_modeling) from the original on 2023-10-10. Retrieved 2023-10-05.

42. "Causal language modeling" (https://huggingface.co/docs/transformers/tasks/language_modeling) . *huggingface.co*. Archived (https://web.archive.org/web/20231010083913/https://huggingface.co/docs/transformers/tasks/language_modeling) from the original on 2023-10-10. Retrieved 2023-10-05.

43. Tay, Yi; Dehghani, Mostafa; Tran, Vinh Q.; Garcia, Xavier; Wei, Jason; Wang, Xuezhi; Chung, Hyung Won; Shakeri, Siamak; Bahri, Dara (2023-02-28). "UL2: Unifying Language Learning Paradigms". arXiv:2205.05131 (https://arxiv.org/abs/2205.05131) [cs.CL (https://arxiv.org/archive/cs.CL)].

44. Team, Keras. "Keras documentation: GPT2Backbone model" (https://keras.io/api/keras_nlp/models/gpt2/gpt2_backbone/) . *keras.io*. Archived (https://web.archive.org/web/20240808085714/https://keras.io/api/keras_nlp/models/gpt2/gpt2_backbone/) from the original on 2024-08-08. Retrieved 2024-08-08.

45. Clark, Kevin; Khandelwal, Urvashi; Levy, Omer; Manning, Christopher D. (August 2019). "What Does BERT Look at? An Analysis of BERT's Attention" (https://www.aclweb.org/anthology/W19-4828) . *Proceedings of the 2019 ACL Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*. Florence, Italy: Association for Computational Linguistics: 276–286. arXiv:1906.04341 (https://arxiv.org/abs/1906.04341) . doi:10.18653/v1/W19-4828 (https://doi.org/10.18653%2Fv1%2FW19-4828) . Archived (https://web.archive.org/web/20201021211357/https://www.aclweb.org/anthology/W19-4828/) from the original on 2020-10-21. Retrieved 2020-05-20.

46. Yang, Zhilin; Dai, Zihang; Yang, Yiming; Carbonell, Jaime; Salakhutdinov, Russ R; Le, Quoc V (2019). "XLNet: Generalized Autoregressive Pretraining for Language Understanding" (https://proceedings.neurips.cc/paper/2019/hash/dc6a7e655d7e5840e66733e9ee67cc69-Abstract.html) . *Advances in Neural Information Processing Systems*. **32**. Curran Associates, Inc. arXiv:1906.08237 (https://arxiv.org/abs/1906.08237) . Archived (https://web.archive.org/web/20240809053521/https://proceedings.neurips.cc/paper/2019/hash/dc6a7e655d7e5840e66733e9ee67cc69-Abstract.html) from the original on 2024-08-09. Retrieved 2024-08-09.

47. Wang, Qiang; Li, Bei; Xiao, Tong; Zhu, Jingbo; Li, Changliang; Wong, Derek F.; Chao, Lidia S. (2019-06-04). "Learning Deep Transformer Models for Machine Translation". arXiv:1906.01787 (https://arxiv.org/abs/1906.01787) [cs.CL (https://arxiv.org/archive/cs.CL)].

48. Phuong, Mary; Hutter, Marcus (2022-07-19). "Formal Algorithms for Transformers". arXiv:2207.09238 (https://arxiv.org/abs/2207.09238) [cs.LG (https://arxiv.org/archive/cs.LG)].

49. Dong, Li; Yang, Nan; Wang, Wenhui; Wei, Furu; Liu, Xiaodong; Wang, Yu; Gao, Jianfeng; Zhou, Ming; Hon, Hsiao-Wuen (2019). "Unified Language Model Pre-training for Natural Language Understanding and Generation" (https://proceedings.neurips.cc/paper_files/paper/2019/hash/c20bb2d9a50d5ac1f713f8b34d9aac5a-Abstract.html) . *Advances in Neural Information Processing Systems*. **32**. Curran Associates, Inc.

50. Gu, Albert; Dao, Tri (2023-12-01). "Mamba: Linear-Time Sequence Modeling with Selective State Spaces". arXiv:2312.00752 (https://arxiv.org/abs/2312.00752) [cs.LG (https://arxiv.org/archive/cs.LG)].

51. Shazeer, Noam (2020-02-01). "GLU Variants Improve Transformer". arXiv:2002.05202 (https://arxiv.org/abs/2002.05202) [cs.LG (https://arxiv.org/archive/cs.LG)].

52. Hendrycks, Dan; Gimpel, Kevin (2016-06-27). "Gaussian Error Linear Units (GELUs)". arXiv:1606.08415v5 (https://arxiv.org/abs/1606.08415v5) [cs.LG (https://arxiv.org/archive/cs.LG)].

53. Zhang, Biao; Sennrich, Rico (2019). "Root Mean Square Layer Normalization" (https://proceedings.neurips.cc/paper/2019/hash/1e8a19426224ca89e83cef47f1e7f53b-Abstract.html) . *Advances in Neural Information Processing Systems*. **32**. Curran Associates, Inc. arXiv:1910.07467 (https://arxiv.org/abs/1910.07467) . Archived (https://web.archive.org/web/20240917113737/https://proceedings.neurips.cc/paper/2019/hash/1e8a19426224ca89e83cef47f1e7f53b-Abstract.html) from the original on 2024-09-17. Retrieved 2024-08-09.

54. Nguyen, Toan Q.; Salazar, Julian (2019-11-02). Niehues, Jan; Cattoni, Rolando; Stüker, Sebastian; Negri, Matteo; Turchi, Marco; Ha, Thanh-Le; Salesky, Elizabeth; Sanabria, Ramon; Barrault, Loic (eds.). "Transformers without Tears: Improving the Normalization of Self-Attention" (https://aclanthology.org/2019.iwslt-1.17) . *Proceedings of the 16th International Conference on Spoken Language Translation*. Hong Kong: Association for Computational Linguistics. arXiv:1910.05895 (https://arxiv.org/abs/1910.05895) . doi:10.5281/zenodo.3525484 (https://doi.org/10.5281%2Fzenodo.3525484) . Archived (https://web.archive.org/web/20240809185028/https://aclanthology.org/2019.iwslt-1.17/) from the original on 2024-08-09. Retrieved 2024-08-09.

55. Dufter, Philipp; Schmitt, Martin; Schütze, Hinrich (September 2022). "Position Information in Transformers: An Overview" (https://doi.org/10.1162%2Fcoli_a_00445) . *Computational Linguistics*. **48** (3): 733–763. arXiv:2102.11090 (https://arxiv.org/abs/2102.11090) . doi:10.1162/coli_a_00445 (https://doi.org/10.1162%2Fcoli_a_00445) .

56. Gehring, Jonas; Auli, Michael; Grangier, David; Yarats, Denis; Dauphin, Yann N. (2017-07-17). "Convolutional Sequence to Sequence Learning" (https://proceedings.mlr.press/v70/gehring17a.html) . *Proceedings of the 34th International Conference on Machine Learning*. PMLR: 1243–1252. Archived (https://web.archive.org/web/20240809184100/https://proceedings.mlr.press/v70/gehring17a.html) from the original on 2024-08-09. Retrieved 2024-08-09.

57. Haviv, Adi; Ram, Ori; Press, Ofir; Izsak, Peter; Levy, Omer (2022-12-05). "Transformer Language Models without Positional Encodings Still Learn Positional Information". arXiv:2203.16634 (https://arxiv.org/abs/2203.16634) [cs.CL (https://arxiv.org/archive/cs.CL)].

58. Kwon, Woosuk; Li, Zhuohan; Zhuang, Siyuan; Sheng, Ying; Zheng, Lianmin; Yu, Cody Hao; Gonzalez, Joseph; Zhang, Hao; Stoica, Ion (2023). "Efficient Memory Management for Large Language Model Serving with PagedAttention". *Proceedings of the 29th Symposium on Operating Systems Principles*. pp. 611–626. doi:10.1145/3600006.3613165 (https://doi.org/10.1145%2F3600006.3613165) . ISBN 979-8-4007-0229-7.

59. "vllm-project/vllm" (https://github.com/vllm-project/vllm) . vLLM. 2024-06-20. Archived (https://web.archive.org/web/20240618204642/https://github.com/vllm-project/vllm) from the original on 2024-06-18. Retrieved 2024-06-20.

60. Zhuohan Li, Woosuk Kwon; Zhuang, Siyuan; Sheng, Ying; Zheng, Lianmin; Yu, Cody; Gonzalez, Joey; Zhang, Hao; Stoica, Ion (2023-06-20). "vLLM: Easy, Fast, and Cheap LLM Serving with PagedAttention" (https://blog.vllm.ai/2023/06/20/vllm.html) . *vLLM Blog*. Archived (https://web.archive.org/web/20240620205208/https://blog.vllm.ai/2023/06/20/vllm.html) from the original on 2024-06-20. Retrieved 2024-06-20.

61. Hu, Cunchen; Huang, Heyang; Xu, Liangliang; Chen, Xusheng; Xu, Jiang; Chen, Shuang; Feng, Hao; Wang, Chenxi; Wang, Sa (2024-01-20). "Inference without Interference: Disaggregate LLM Inference for Mixed Downstream Workloads". arXiv:2401.11181 (https://arxiv.org/abs/2401.11181) [cs.DC (https://arxiv.org/archive/cs.DC)].

62. Yu, Gyeong-In; Jeong, Joo Seong; Kim, Geon-Woo; Kim, Soojeong; Chun, Byung-Gon (2022). *Orca: A Distributed Serving System for Transformer-Based Generative Models* (https://www.usenix.org/conference/osdi22/presentation/yu) . pp. 521–538. ISBN 978-1-939133-28-1.

63. Dettmers, Tim; Lewis, Mike; Belkada, Younes; Zettlemoyer, Luke (2022-08-15). "LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale". arXiv:2208.07339v2 (https://arxiv.org/abs/2208.07339v2) [cs.LG (https://arxiv.org/archive/cs.LG)].

64. Dao, Tri; Ermon, Stefano; Fu, Dan; Ré, Christopher; Rudra, Atri (2022). "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness". *Advances in Neural Information Processing Systems 35*. pp. 16344–16359. arXiv:2205.14135 (https://arxiv.org/abs/2205.14135) . doi:10.52202/068431-1189 (https://doi.org/10.52202%2F068431-1189) . ISBN 978-1-7138-7108-8.

65. Milakov, Maxim; Gimelshein, Natalia (2018). "Online normalizer calculation for softmax". arXiv:1805.02867 (https://arxiv.org/abs/1805.02867) [cs.PF (https://arxiv.org/archive/cs.PF)].

66. "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning" (https://crfm.stanford.edu/2023/07/17/flash2.html) . *crfm.stanford.edu*. Archived (https://web.archive.org/web/20230718103613/https://crfm.stanford.edu/2023/07/17/flash2.html) from the original on 2023-07-18. Retrieved 2023-07-18.

67. Chowdhery, Aakanksha; Narang, Sharan; Devlin, Jacob; Bosma, Maarten; Mishra, Gaurav; Roberts, Adam; Barham, Paul; Chung, Hyung Won; Sutton, Charles; Gehrmann, Sebastian; Schuh, Parker; Shi, Kensen; Tsvyashchenko, Sasha; Maynez, Joshua; Rao, Abhishek (2022-04-01). "PaLM: Scaling Language Modeling with Pathways". arXiv:2204.02311 (https://arxiv.org/abs/2204.02311) [cs.CL (https://arxiv.org/archive/cs.CL)].

68. Ainslie, Joshua; Lee-Thorp, James; de Jong, Michiel; Zemlyanskiy, Yury; Lebrón, Federico; Sanghai, Sumit (2023-12-23). "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints". arXiv:2305.13245 (https://arxiv.org/abs/2305.13245) [cs.CL (https://arxiv.org/archive/cs.CL)].

69. DeepSeek-AI; Liu, Aixin; Feng, Bei; Wang, Bin; Wang, Bingxuan; Liu, Bo; Zhao, Chenggang; Dengr, Chengqi; Ruan, Chong (19 June 2024). "DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model". arXiv:2405.04434 (https://arxiv.org/abs/2405.04434) [cs.CL (https://arxiv.org/archive/cs.CL)]..

70. Leviathan, Yaniv; Kalman, Matan; Matias, Yossi (2023-05-18). "Fast Inference from Transformers via Speculative Decoding". arXiv:2211.17192 (https://arxiv.org/abs/2211.17192) [cs.LG (https://arxiv.org/archive/cs.LG)].

71. Fu, Yao (2023-12-11). "Towards 100x Speedup: Full Stack Transformer Inference Optimization" (https://yaofu.notion.site/Towards-100x-Speedup-Full-Stack-Transformer-Inference-Optimization-43124c3688e14cffaf2f1d6cbdf26c6c) . *yaofu.notion.site*.

72. Chen, Charlie; Borgeaud, Sebastian; Irving, Geoffrey; Lespiau, Jean-Baptiste; Sifre, Laurent; Jumper, John (2023-02-02). "Accelerating Large Language Model Decoding with Speculative Sampling". arXiv:2302.01318 (https://arxiv.org/abs/2302.01318) [cs.CL (https://arxiv.org/archive/cs.CL)].

73. Gloeckle, Fabian; Badr Youbi Idrissi; Rozière, Baptiste; Lopez-Paz, David; Synnaeve, Gabriel (2024). "Better & Faster Large Language Models via Multi-token Prediction". arXiv:2404.19737 (https://arxiv.org/abs/2404.19737) [cs.CL (https://arxiv.org/archive/cs.CL)].

74. DeepSeek-AI; et al. (2024). "DeepSeek-V3 Technical Report". arXiv:2412.19437 (https://arxiv.org/abs/2412.19437) [cs.CL (https://arxiv.org/archive/cs.CL)].

75. Kitaev, Nikita; Kaiser, Łukasz; Levskaya, Anselm (2020). "Reformer: The Efficient Transformer". arXiv:2001.04451 (https://arxiv.org/abs/2001.04451) [cs.LG (https://arxiv.org/archive/cs.LG)].

76. Liu, Ze; Lin, Yutong; Cao, Yue; Hu, Han; Wei, Yixuan; Zhang, Zheng; Lin, Stephen; Guo, Baining (2021). "Swin Transformer: Hierarchical Vision Transformer using Shifted Windows". *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*. IEEE. pp. 9992–10002. arXiv:2103.14030 (https://arxiv.org/abs/2103.14030) . doi:10.1109/ICCV48922.2021.00986 (https://doi.org/10.1109%2FICCV48922.2021.00986) . ISBN 978-1-6654-2812-5.

77. Ristea, Nicolaea Catalin; Ionescu, Radu Tudor; Khan, Fahad Shahbaz (2022-09-18). "SepTr: Separable Transformer for Audio Spectrogram Processing" (https://www.isca-archive.org/interspeech_2022/ristea22_interspeech.html) . *Interspeech*. ISCA: 4103–4107. arXiv:2203.09581 (https://arxiv.org/abs/2203.09581) . doi:10.21437/Interspeech.2022-249 (https://doi.org/10.21437%2FInterspeech.2022-249) .

78. Tay, Yi; Dehghani, Mostafa; Abnar, Samira; Shen, Yikang; Bahri, Dara; Pham, Philip; Rao, Jinfeng; Yang, Liu; Ruder, Sebastian; Metzler, Donald (2020-11-08). "Long Range Arena: A Benchmark for Efficient Transformers". arXiv:2011.04006 (https://arxiv.org/abs/2011.04006) [cs.LG (https://arxiv.org/archive/cs.LG)].

79. Child, Rewon; Gray, Scott; Radford, Alec; Sutskever, Ilya (2019-04-23). "Generating Long Sequences with Sparse Transformers". arXiv:1904.10509 (https://arxiv.org/abs/1904.10509) [cs.LG (https://arxiv.org/archive/cs.LG)].

80. "Constructing Transformers For Longer Sequences with Sparse Attention Methods" (https://ai.googleblog.com/2021/03/constructing-transformers-for-longer.html) . *Google AI Blog*. 25 March 2021. Archived (https://web.archive.org/web/20210918150757/https://ai.googleblog.com/2021/03/constructing-transformers-for-longer.html) from the original on 2021-09-18. Retrieved 2021-05-28.

81. Lu, Kevin; Grover, Aditya; Abbeel, Pieter; Mordatch, Igor (2022). "Frozen Pretrained Transformers as Universal Computation Engines". *Proceedings of the AAAI Conference on Artificial Intelligence*. **36** (7): 7628–7636. doi:10.1609/aaai.v36i7.20729 (https://doi.org/10.1609%2Faaai.v36i7.20729) .

82. "Vicuna: An Open-Source Chatbot Impressing GPT-4 with 90%* ChatGPT Quality | LMSYS Org" (https://lmsys.org/blog/2023-03-30-vicuna) . *lmsys.org*. 30 March 2023. Archived (https://web.archive.org/web/20240812055843/https://lmsys.org/blog/2023-03-30-vicuna/) from the original on 2024-08-12. Retrieved 2024-08-11.

83. Liu, Haotian; Li, Chunyuan; Wu, Qingyang; Lee, Yong Jae (2023-12-15). "Visual Instruction Tuning" (https://proceedings.neurips.cc/paper_files/paper/2023/hash/6dcf277ea32ce3288914faf369fe6de0-Abstract-Conference.html) . *Advances in Neural Information Processing Systems*. Vol. 36. pp. 34892–34916. doi:10.52202/075280-1516 (https://doi.org/10.52202%2F075280-1516) . ISBN 978-1-7138-9911-2. Archived (https://web.archive.org/web/20240926183648/https://proceedings.neurips.cc//paper_files/paper/2023/hash/6dcf277ea32ce3288914faf369fe6de0-Abstract-Conference.html) from the original on 2024-09-26. Retrieved 2024-08-11.

84. Radford, Alec; Kim, Jong Wook; Xu, Tao; Brockman, Greg; McLeavey, Christine; Sutskever, Ilya (2022). "Robust Speech Recognition via Large-Scale Weak Supervision". arXiv:2212.04356 (https://arxiv.org/abs/2212.04356) [eess.AS (https://arxiv.org/archive/eess.AS)].

85. Jaegle, Andrew; Gimeno, Felix; Brock, Andrew; Zisserman, Andrew; Vinyals, Oriol; Carreira, Joao (2021-06-22). "Perceiver: General Perception with Iterative Attention". arXiv:2103.03206 (https://arxiv.org/abs/2103.03206) [cs.CV (https://arxiv.org/archive/cs.CV)].

86. Jaegle, Andrew; Borgeaud, Sebastian; Alayrac, Jean-Baptiste; Doersch, Carl; Ionescu, Catalin; Ding, David; Koppula, Skanda; Zoran, Daniel; Brock, Andrew; Shelhamer, Evan; Hénaff, Olivier (2021-08-02). "Perceiver IO: A General Architecture for Structured Inputs & Outputs". arXiv:2107.14795 (https://arxiv.org/abs/2107.14795) [cs.LG (https://arxiv.org/archive/cs.LG)].

87. "Parti: Pathways Autoregressive Text-to-Image Model" (https://sites.research.google/parti/) . *sites.research.google*. Archived (https://web.archive.org/web/20240810094406/https://sites.research.google/parti/) from the original on 2024-08-10. Retrieved 2024-08-09.

88. Villegas, Ruben; Babaeizadeh, Mohammad; Kindermans, Pieter-Jan; Moraldo, Hernan; Zhang, Han; Saffar, Mohammad Taghi; Castro, Santiago; Kunze, Julius; Erhan, Dumitru (2022-09-29). "Phenaki: Variable Length Video Generation from Open Domain Textual Descriptions". arXiv:2210.02399 (https://arxiv.org/abs/2210.02399) [cs.CV (https://arxiv.org/archive/cs.CV)].

89. Chang, Huiwen; Zhang, Han; Barber, Jarred; Maschinot, A. J.; Lezama, Jose; Jiang, Lu; Yang, Ming-Hsuan; Murphy, Kevin; Freeman, William T. (2023-01-02). "Muse: Text-To-Image Generation via Masked Generative Transformers". arXiv:2301.00704 (https://arxiv.org/abs/2301.00704) [cs.CV (https://arxiv.org/archive/cs.CV)].

90. Ramesh, Aditya; Pavlov, Mikhail; Goh, Gabriel; Gray, Scott; Voss, Chelsea; Radford, Alec; Chen, Mark; Sutskever, Ilya (2021-02-26). "Zero-Shot Text-to-Image Generation". arXiv:2102.12092 (https://arxiv.org/abs/2102.12092) [cs.CV (https://arxiv.org/archive/cs.CV)].

91. Yu, Jiahui; Xu, Yuanzhong; Koh, Jing Yu; Luong, Thang; Baid, Gunjan; Wang, Zirui; Vasudevan, Vijay; Ku, Alexander; Yang, Yinfei (2022-06-21). "Scaling Autoregressive Models for Content-Rich Text-to-Image Generation". arXiv:2206.10789 (https://arxiv.org/abs/2206.10789) [cs.CV (https://arxiv.org/archive/cs.CV)].

92. Kariampuzha, William; Alyea, Gioconda; Qu, Sue; Sanjak, Jaleal; Mathé, Ewy; Sid, Eric; Chatelaine, Haley; Yadaw, Arjun; Xu, Yanji; Zhu, Qian (2023). "Precision information extraction for rare disease epidemiology at scale" (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC9972634) . *Journal of Translational Medicine*. **21** (1): 157. doi:10.1186/s12967-023-04011-y (https://doi.org/10.1186%2Fs12967-023-04011-y) . PMC 9972634 (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC9972634) . PMID 36855134 (https://pubmed.ncbi.nlm.nih.gov/36855134) .

93. Hsu, Cyril Shih-Huan; Dalgkitsis, Anestis; Grosso, Paola; Papagianni, Chrysa (2026). "Transformer-Empowered Actor-Critic Reinforcement Learning for Sequence-Aware Service Function Chain Partitioning". *IEEE Transactions on Network Science and Engineering*. **13**: 9247–9264. arXiv:2504.18902 (https://arxiv.org/abs/2504.18902) . Bibcode:2026ITNSE..13.9247H (https://scixplorer.org/abs/2026ITNSE..13.9247H) . doi:10.1109/TNSE.2026.3689920 (https://doi.org/10.1109%2FTNSE.2026.3689920) .

Дополнительная литература

- Александр Раш, «Аннотированный трансформер» (https://nlp.seas.harvard.edu/2018/04/03/attention.html) Архивировано (https://web.archive.org/web/20210922093841/https://nlp.seas.harvard.edu/2018/04/03/attention.html) 22 сентября 2021 года в Wayback Machine, Гарвардская группа по изучению нейролингвистики, 3 апреля 2018 года
- США патент 10452978 (https://worldwide.espacenet.com/textdoc?DB=EPODOC&IDX=US10452978) , Ноам М. Шазир; Эйдан Николас Гомес; Лукаш Мечислав Кайзер; Якоб Д. Ушкорайт; Ллион Оуэн Джонс; Ники Дж. Пармар; Илья Полосухин; Ашиш Теку Васвани, "Нейронные сети для преобразования последовательностей на основе внимания", выдан 22 октября 2019 года, присвоен компании Google LLC
- Фуонг, Мэри; Хаттер, Маркус (2022). «Формальные алгоритмы для трансформеров». arXiv:2207.09238 (https://arxiv.org/abs/2207.09238) [cs.LG (https://arxiv.org/archive/cs.LG)].